

Лабораторна робота № 12

Мета: Ознайомлення з поведінковими патернами проектування, практичне застосування патернів та їх реалізація в обраній мові програмування.

Завдання:

ЗАВДАННЯ 1

Відповідно до варіанту (додаток А, патерн 1) та предметної області (додаток В) реалізувати приклад застосування поведінкового патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 2 Базове розгортання проекту на віддаленому сервері

Відповідно до варіанту (додаток А, патерн 2) та предметної області (додаток В) реалізувати приклад застосування поведінкового патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 3

Відповідно до варіанту (додаток А, патерн 3) та предметної області (додаток В) додати до попередньої реалізації WebApi з ЛР4-6 застосування поведінкового патерну проектування.

Хід роботи:

Завдання 1

Реалізовано виконання завдання із використанням поведінкового патерну Chain of Responsibility:

Створено модель користувача:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Models
{
    public class User
    {
        public List<Sub> subs { get; private set; }
        public string Name { get; set; }
        public double Balance { get; set; }
    }
}
```

					ЛР.ОК24.ПІ231.03.12		
Змін.	Аркуш	№ докум.	Підпис	Дата	Behavioral Design Patterns		
Розробив	Башук О. Ю.						
Перевірів	Жереб Д. В.						
Н.контр.							
Затвер.							
					Літ	Аркуш	Аркушів
					н	1	8
					Х П К		

```

public bool IsBlocked { get; set; }
public User(string name, double balance, bool isBlocked)
{
    Name = name;
    Balance = balance;
    IsBlocked = isBlocked;
    subs = new List<Sub>();
}
public void AddSub(Sub sub) =>
    subs.Add(sub);
public void RemoveSub(Sub sub)
{
    if (!subs.Contains(sub))
        throw new ArgumentException();
    subs.Remove(sub);
}
}
}

```

Створено модель підписки:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Models
{
    public class Sub
    {
        public string Service { get; set; }
        public string Type { get; set; }
        public double Cost { get; set; }

        public Sub(string service, string type, double cost)
        {
            Service = service;
            Type = type;
            Cost = cost;
        }
    }
}

```

Створено абстрактний клас для реалізації потоку обробки, під час використання його як батьківського:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Models;
namespace Task1.Abstract
{
    public abstract class SubHandler

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						2
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    {
        protected SubHandler Next;
        public SubHandler SetNext(SubHandler next)
        {
            Next = next;
            return next;
        }
        public abstract void Handle(User user, Sub sub);
    }
}

```

Створено клас BalanceHandler, що наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class BalanceHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(user.Balance < sub.Cost)
            {
                Console.WriteLine("Недостатньо коштів");
                return;
            }
            Console.WriteLine("Баланс відповідає потребам");
            if(Next is not null)
                Next.Handle(user, sub);
        }
    }
}

```

Створено клас UserBlockHandler, що наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class UserBlockHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(user.IsBlocked)
            {

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						3
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        Console.WriteLine("Користувач заблокований");
        return;
    }
    Console.WriteLine("Користувач не заблокований");
    if(Next is not null)
        Next.Handle(user, sub);
    }
}

```

Створено клас SubTypeHandler, що наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class SubTypeHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(sub.Type != "Standart" || sub.Type == "Premium")
            {
                Console.WriteLine($"Невірний тип підписки: {sub.Type}");
                return;
            }
            Console.WriteLine("Вірний тип підписки");
            if(Next is not null)
                Next.Handle(user, sub);
        }
    }
}

```

Створено фінальний клас в ланцюгу викликів, який також наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class FinalSteepHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            user.Balance -= sub.Cost;
            user.AddSub(sub);
        }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						4
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        Console.WriteLine($"Підписка оформлена для користувача: {user.Name}");
    }
}
}

Організовано головну програму:
using System.Text;
using Task1.Abstract;
using Task1.Handlers;
using Task1.Models;
public class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        User user = new User("User",1000,false);
        Sub sub = new Sub("Netflix","Standart",500);
        SubHandler handler = new UserBlockHandler();
        handler.SetNext(new BalanceHandler())
            .SetNext(new SubTypeHandler())
            .SetNext(new FinalSteepHandler());
        handler.Handle(user, sub);
        Console.WriteLine($"Результат додавання підписки: {user.subs[0].Service} присутня"
    }
}

```

Протестовано роботу програми:

```

Користувач не заблокований
Баланс відповідає потребам
Вірний тип підписки
Підписка оформлена для користувача: User
Результат додавання підписки: Netflix присутня

```

Рисунок 1 – Результат виконання програми

Завдання 2

Рналізовано виконання завдання за допомогою використання паттерну – посередник:

Створено інтерфейс для посередника:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Interfaces
{
    public interface ISubMeddiator
    {
        public void Subscribe(User user, Sub sub);
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						5
Вим.	Арк.	№ докум.	Підпис	Дата		

Створено клас – посередник, що реалізує вказаний інтерфейс:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Interfaces;
using Task2.Models;
using Task2.Parts;
namespace Task2
{
    public class SubMediator : ISubMediator
    {
        private Payment _paymentService;
        private Notification _notification;
        public SubMediator(Payment payment, Notification notification)
        {
            _paymentService = payment;
            _notification = notification;
        }
        public void Subscribe(User user, Sub sub)
        {
            Logger.Instance.Log($"Користувач {user.Name} хоче оформити підписку {sub.Name}");
            if (!_paymentService.Pay(user, sub))
            {
                _notification.Send(user, "не достатньо коштів для оформлення підписки");
                Logger.Instance.Log("Помилка, недостатньо коштів");
                return;
            }
            user.subs.Add(sub);
            _notification.Send(user, "підписку оформлено");
            Logger.Instance.Log("Успіх, підписка оформлена");
        }
    }
}

```

Створено частини, які будуть взаємодіяти за допомогою посередника:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Parts
{
    public class Notification
    {
        public void Send(User user, string message)
        {
            Console.WriteLine($"Повідомлення до {user.Name}: {message}");
        }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						6
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Parts
{
    public class Payment
    {
        public bool Pay(User user, Sub sub)
        {
            if(user.Balance < sub.Price)
                return false;
            user.Balance -= sub.Price;
            return true;
        }
    }
}

```

Створено моделі об'єктів, які беруть участь у прямій роботі програми:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task2.Models
{
    public class User
    {
        public string Name { get; set; }
        public double Balance { get; set; }
        public List<Sub> subs { get; } = new List<Sub>();
        public User(string name, double balance)
        {
            Name = name;
            Balance = balance;
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task2.Models
{
    public class Sub
    {
        public string Name { get; set; }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						7
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        public double Price { get; set; }
        public Sub(string name, double price)
        {
            Name = name; Price = price;
        }
    }
}

```

Розроблено головну програму:

```

using System.Text;
using Task2;
using Task2.Interfaces;
using Task2.Models;
using Task2.Parts;
public class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        User user = new User("User1", 1000);
        Sub sub = new Sub("Netflix", 500);
        ISubMeddiator subMeddiator = new SubMediator(new Payment(), new Notification());
        subMeddiator.Subscribe(user, sub);
    }
}

```

Протестовано роботу головної програми:

Повідомлення до User1: підписку оформлено

Рисунок 2 – Результат виконання програми

ВИСНОВОК:

На даній лабораторній вивчив основи роботи із поведінковими паттернами, отримав практичні навички роботи із паттернами: Chain of Responsibility та посередник

					ЛР.ОК24.ПІ231.03.12	Арк.
						8
Вим.	Арк.	№ докум.	Підпис	Дата		